COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	10/16/2026	**DATE SUBMITTED:**	10/21/2026
NAME:	Juan Paulo C. Lara Mencidor, Renverlanz D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13.1 Parallel Algorithms and Multithreading

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A & B

Screenshot

```cpp
#include <iostream>
#include <thread>
#include <string>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");

    t1.join();

    return 0;
}
```

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis

(Lara)
Running this code, the main function creates and initializes thread “t1”. It then passes three arguments, which are the function name, integer value, and a string of text. This function called by this thread prints the integer and string values it passed to this function, where the console outputs show the given integer 10 and string “T.I.P.”. The join command at the main function waits for “t1” to finish its task to prevent an early termination, which could potentially result in a runtime crash.
(Mencidor)
In this C++ program, the main function spawns a thread named “t1” and passes the function “print” along with two arguments: the integer “10” and the string literal “T.I.P”. The pint function receives these arguments and displays them to the console producing the given output. Finally, calling “t1.join()” in main ensures the main execution thread waits for t1 to finish executing

	before exiting. This prevents premature program termination and avoids potential runtime errors or crashes.

Table 13-1. Simple One-Threaded Example

Screenshot

```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <string>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };

    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }

    return 0;
}
```

Outputs

```
1 : Competent0 : T.I.P.
3 : Engineers
2 : Computer
```

```
0 : T.I.P.2 : Computer
1 : Competent
3 : Engineers
```

Analysis

(Lara)
In this multi-threaded program, there are two for loops: One performs the thread initializations for four concurrent threads based on the index of the array of strings. The next performs iterations on the threads vector and invokes .join() for each thread, which waits for each thread to complete their respective tasks. In contrast to the previous example, the console outputs show different arrangements of threads printing their assigned strings due to the nature of multiple threads performing one task concurrently.
(Mencidor)
This multi-threaded program uses two “for loops”: first, initializes four concurrent threads to

	process a vector of strings by passing the “print” function, the loop index, and the string element, while the Second calls “.join()” on each thread so the main thread waits for completion. Unlike single threaded execution, the sample outputs show varying line orders and interleaved text because the concurrent threads are scheduled non-deterministically by the operating system.

Table 13-2. Multithreaded Example.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

PART A

1. Write a definition of multithreading and its advantages/disadvantages.

(Lara)
Multithreading in C++ executes sub-tasks independently using thread objects like “std::thread”. Its advantages include efficient CPU core utilization through thread pools and lower memory overhead than creating full processes. However, its main disadvantages are data races when threads access shared memory simultaneously and unpredictable execution order unless synchronized with tools like mutexes.

(Mencidor)
Multithreading breaks a program into smaller units running concurrently alongside the main thread. Its key benefits are high performance on complex applications and better user interface responsiveness by keeping worker tasks in the background. Conversely, creating individual threads for short tasks creates unnecessary overhead, and unmanaged thread interaction causes race conditions and corrupted data.

2. Rationalize the use of multithreading by providing at least 3 use-cases.

(Lara)
Multithreading is essential for optimizing system workload and efficiency. Three clear use-cases include the Parallel Batch Processing such as formatting or printing string datasets via thread loops. The Managed Arithmetic Calculations for distributing operations across dedicated worker thread pools without overloading resources. And lastly, Non-Blocking Background Work for offloading heavy functions to secondary threads so the main application thread stays responsive.

(Mencidor)
Multithreading provides significant performance gains across data heavy applications. Key use cases include the Asynchronous Task Execution, where background tasks run on separate threads while “join()” coordinates safe completion. The Independent Variable Operations that runs concurrent calculations on distinct data without dependency issues. And, Multicore Vector Processing for distributing vector or array elements across threads to reduce total runtime.

3. Differentiate between parallelism and concurrency.

(Lara)
Concurrency overlaps multiple tasks over a single shared execution flow where threads interact through shared memory. Parallelism physically executes separate copies of a program at the exact same instant across different CPU cores without requiring thread communication. In concept, concurrency is like having two queues share one espresso machine, whereas parallelism is like having two queues use two separate espresso machines

simultaneously.

(Mencidor)
Concurrency focuses on task management and interleaving complex, interdependent operations that require synchronization tools like mutexes to avoid race conditions. Parallelism focuses purely on raw speed by executing independent data streams at once on multi core hardware. IN C++, basic concurrency features (threads, locks) arrived in C++11, while parallel algorithms were added later 2017

PART B

Code

```cpp
#include <iostream>
#include <thread>

// a global variable of type integer
int global_var = 0;

// add function to add an int value to the global variable
void add(int value) {
    global_var += value;
}

int main() {
    // create 3 threads to pass the add function to the threads
    std::thread t1(add, 10);
    std::thread t2(add, 20);
    std::thread t3(add, 30);

    // display global var
    std::cout << "Initial Display (Before t1.join): global_var = " << global_var << std::endl;

    // combinations of join() and display
    t1.join();
    std::cout << "Display after t1.join(): global_var = " << global_var << std::endl;
    t2.join();
    std::cout << "Display after t2.join(): global_var = " << global_var << std::endl;
    t3.join();
    std::cout << "Display after t3.join(): global_var = " << global_var << std::endl;

    return 0;
}
```

Outputs

```
Initial Display (Before t1.join): global_var = 0
Display after t1.join(): global_var = 60
Display after t2.join(): global_var = 60
Display after t3.join(): global_var = 60
```

Analysis

(Lara)
In the code, three threads are initialized to pass the add function to the thread. The global variable is checked before invoking the join() to show the initial value and indicate if the running threads are not finished in executing the add function. Calling the join function causes the main thread to block and pause until each of the threads are done working. It synchronizes the threads and

	allows one thread to safely execute and stop for the next thread until all threads are complete in executing the function, which prevents race conditions and runtime errors. (Mencidor) When a thread is launched, it runs asynchronously alongside the main thread. Checking “global counter” before calling “.join()” shows its initial value because the worker thread hasn’t finished running “add() yet. Calling “.join()” pauses the main thread until the worker finishes, ensuring the update completes safely and preventing data races on shared memory. Spawning all threads simultaneously without “.join()” or locks would cause race conditions and unpredictable results.

PART C

Code

```cpp
#include <iostream>
#include <thread>
#include <chrono>

const int SIZE = 500000;

// merge two sorted sub-arrays
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // sub-arrays
    int* L = new int[n1];
    int* R = new int[n2];

    for (int i = 0; i < n1; i++){
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; j++){
        R[j] = arr[mid + 1 + j];
    }
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }
    while (i < n1){
        arr[k++] = L[i++];
    }
    while (j < n2) {
        arr[k++] = R[j++];
    }
}
```

```cpp
// Multi-threaded Merge Sort
void mergeSortParallel(int arr[], int left, int right, int depth = 0) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        // Limit recursive thread depth
        if (depth < 2) {
            std::thread t1(mergeSortParallel, arr, left, mid, depth + 1);
            std::thread t2(mergeSortParallel, arr, mid + 1, right, depth + 1);
            t1.join();
            t2.join();
        } else {
            mergeSortSequential(arr, left, mid);
            mergeSortSequential(arr, mid + 1, right);
        }
        merge(arr, left, mid, right);
    }
}
```

```cpp
int main() {
    int* arr1 = new int[SIZE];
    int* arr2 = new int[SIZE];

    // Populate arr1 and duplicate to arr2
    for (int i = 0; i < SIZE; i++) {
        arr1[i] = SIZE - i;
        arr2[i] = arr1[i];
    }
    // EXECUTION TIME using chrono clock
    // Single-threaded
    auto startSeq = std::chrono::high_resolution_clock::now();
    mergeSortSequential(arr1, 0, SIZE - 1);
    auto endSeq = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> seqTime = endSeq - startSeq;
    // Multi-threaded
    auto startPar = std::chrono::high_resolution_clock::now();
    mergeSortParallel(arr2, 0, SIZE - 1);
    auto endPar = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> parTime = endPar - startPar;

    std::cout << "Data Size:    " << SIZE << " elements\n";
    std::cout << "Single Threaded Merge Sort Time:    " << seqTime.count() << " ms\n";
    std::cout << "Multi Threaded Merge Sort Time:     " << parTime.count() << " ms\n";

    return 0;
}
```

Outputs

```
Data Size:    500000 elements
Single Threaded Merge Sort Time:    72.5474 ms
Multi Threaded Merge Sort Time:     33.8863 ms
```

Analysis

(Lara)

	By applying multi-threading to the Merge Sort algorithm, the array of 500,000 elements is split at the top recursive levels and assigned to the working threads. Restricting the recursion depth prevents thread over saturation, which allows sub arrays to sort in parallel across available CPU cores before being merged sequentially. As seen in the outputs, the execution time dropped significantly from 72.5 ms to 33.9 ms, which shows an increase of speed at roughly 2x. This demonstrates the noticeable performance boost provided by multi-threading for heavy sorting tasks. (Mencidor) In this implementation, multi-threading optimizes Merge Sort by dividing the data set and executing recursive sorting tasks across separate threads up to a specified depth limit. Calling ".join()" on t1 and t2 ensures that both sub arrays are completely sorted before the final "merge()" operation combines them. The benchmark demonstrates a nearly 2x speedup, reducing the sorting time from roughly 72.5 ms down to 33.9 ms. This confirms that parallel execution effectively cuts down processing overhead for large data structures, provided that thread creation depth is controlled to avoid resource exhaustion.

CONCLUSION

(Lara)
In conclusion, this Hands-on Activity has introduced the concepts of Parallelism and Multithreading by demonstrating the differences between concurrency and parallelism to show how threads are used to perform multiple tasks at the same time while synchronizing these elements with the thread join function as shown in the procedures. The activities apply these concepts into working code implementations that show the basic way of how multi-threading in code works. It also illustrates how concurrency works in code with the second table showing the unpredictable arrangement of the printed strings in the multi-threaded implementation. The supplementary activities apply the learnings from the procedures by integrating multi-threading to algorithms used in previous activities, such as the merge sorting algorithm. This implementation significantly cuts down the time that this more efficient algorithm already takes into half, which demonstrates the applications of multi-threading learned in this lab activity.

(Mencidor)
Doing this activity gave me a hands-on feel for multithreading and concurrency with the "std::thread" code. It was nice to see how delegating tasks across threads and properly syncing them with the ".join()" code keeps things running smoothly without running into messy data races. Step by step, the procedure made it easy to see how moving from standard single threaded code to multi threaded loops changes the flow of execution. The supp. activities really brought it to home, showing how join points affect shared global variables and how splitting data across threads can give big dataset algorithms a noticeable speed boost. I feel like I did a good job putting these programs together and making sense of the execution timing.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts and clarification of instructions. Assistance in parts of the supplementary activities and grammar correction in the analyses.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to clarify certain instructions that were difficult to understand, and explain the concepts discussed in the activity and lectures. It was also used to clarify certain points in the supplementary activities to ensure the answers and analyses are correct.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.